

Introducing dynamic diversity into a discrete particle swarm optimization

Alberto García-Villoria, Rafael Pastor*

*Institute of Industrial and Control Engineering (IOC), Technical University of Catalonia (UPC), Av. Diagonal 647
(Edif. ETSEIB), 11th floor, 08028 Barcelona, Spain*

Available online 28 January 2008

Abstract

Particle swarm optimization (PSO) is an evolutionary metaheuristic inspired by the flocking behaviour of birds, which has successfully been used to solve several kinds of problems, although there are few studies aimed at solving discrete optimization problems. One disadvantage of PSO is the risk of a premature search convergence. To prevent this, we propose to introduce diversity into a discrete PSO by adding a *random velocity*. The degree of the introduced diversity is not static (i.e. preset before running PSO) but instead changes dynamically according to the heterogeneity of the population (i.e. if the search has converged or not). We solve the response time variability problem (RTVP) to test these two new ideas. The RTVP is an NP-hard combinatorial scheduling problem that has recently appeared in the literature. It occurs whenever products, clients or jobs need to be sequenced in such a way that the variability in the time between the instants at which they receive the necessary resources is minimized. The most efficient algorithm for solving non-small instances of the RTVP published to date is a classical PSO algorithm, referred to by the authors as *PSO-MIF*. In this paper, we propose 10 discrete PSO algorithms for solving the RTVP: one based on the ideas described above (*PSO-c₃dyn*) and nine based on strategies proposed in the literature and adapted for solving a discrete optimization problem such as the RTVP. We compare all 11 PSO algorithms and the computational experiment shows that, on average, the best results obtained are due to our proposal of dynamic control mechanism for introducing diversity.

© 2007 Elsevier Ltd. All rights reserved.

Keywords: Particle swarm optimization; Adaptive parameters; Response time variability; Scheduling

1. Introduction

Particle swarm optimization (PSO) is an evolutionary stochastic metaheuristic designed by Kennedy and Eberhart [1] that has been studied by several researchers since its publication [2]. PSO has been successfully applied to a variety of problems such as artificial neural network training [3,4], combinatorial optimization problems [5–8] and multiobjective optimization problems [9,10]. Most of the PSO applications published in the literature were designed to solve continuous optimization problems, but there are few PSO applications for discrete optimization problems. The applications of PSO on combinatorial optimization problems are still considered limited, but the advantages of PSO include a simple structure, immediately accessible for practical applications, easy of implementation, speed to acquire solutions and robustness [11]. In this paper, we develop 10 discrete PSO algorithms for solving an NP-hard scheduling problem.

* Corresponding author. Tel.: +34 93 401 17 01; fax: +34 93 401 66 05.

E-mail addresses: alberto.garcia-villoria@upc.edu (A. García-Villoria), rafael.pastor@upc.edu (R. Pastor).

The core of PSO is based on an analogy of the social behaviour of flocks of birds when they search for food. Since it is a population-based evolutionary metaheuristic, PSO has a population (known as swarm in the PSO ambit) of particles. Each particle has an associated point in the search space (which represents a solution) and an associated velocity (which indicates how the point of the particle is moved in the search space). The current velocity of a particle is typically a linear combination of three types of velocity: (1) the inertia velocity (i.e. its previous velocity); (2) the velocity to the best point found by the particle and (3) the velocity to the best point found by the swarm. The PSO algorithm iteratively modifies the point and the velocity of each particle as it looks for the optimal solution.

The trade-off between the exploration (i.e. the global search) and the exploitation (i.e. the local search) of the search space is critical to the success of an evolutionary metaheuristic. Trelea [12] demonstrated that PSO always converges at certain values of its parameters but if the convergence is premature then several regions of the search space will remain unexplored. Several strategies have been proposed in the literature for correcting the tendency to converge prematurely on a local optimum.

Clerc [13] suggested a PSO variant in which the velocity to the best point found by the swarm is replaced by the velocity to the current best point of the swarm, although he does not test this variant. The idea was later implemented in a discrete PSO for solving the flowshop problem [6].

Several authors developed strategies based on dynamically modifying the value of the PSO parameter called *inertia weight*, which weights the inertia velocity. Larger inertia weight values facilitate a more global behaviour and smaller values facilitate a more local behaviour. Therefore, the inertia weight is changed to achieve better balance dynamics between the global and local search capabilities. Since exploration is more important at the beginning of the search process and exploitation is more important at the end, the usual strategy is to start with a large inertia weight value and then decrease it over the iterations of the algorithm [14], which is the method adopted by Eberhart and Shi [15], He et al. [16] and Shi and Eberhart [17].

Other authors introduce diversity into the swarm to escape from the current local optimum. There are different ways of introducing diversity and controlling the degree of diversity introduced. Clerc [18] and Zhang et al. [19] dynamically change the size of the swarm according to the performance of the algorithm. The size of the swarm is important because too few particles will cause the algorithm to converge prematurely to a local optimum, while too many particles will slow down the algorithm. Note that adding new particles when the algorithm converges introduces diversity into the swarm. He et al. [16] developed a PSO variant, called PSO passive congregation (PSOPC), which was inspired by the work of Parrish and Hamner [20] in which spatial structure of animal group organizations is modelled. PSOPC uses an analogy of passive congregation, which describes a situation in which an individual is attracted to other group members but there is no display of social behaviour. Passive congregation is expressed in PSOPC by adding a new term to the velocity at which the particle tends to move towards the point of another particle in the swarm that is randomly selected at each iteration of the PSOPC algorithm. This new term is weighted by a factor which is initially set to a small value and then linearly increased over the iterations of the algorithm. Xie et al. [21] propose two dissipative PSO (DPSO) algorithms: in the first, for each particle some *components* of the velocity are chosen at random and weighted by a random weight that is uniformly distributed between 0 and 1; in the other, for each particle some *components* of the point are chosen at random and weighted by a random weight that is uniformly distributed between 0 and 1. Fieldsend and Singh [22] add a new term to the velocity called *turbulence*, which represents a random velocity. This idea was initially proposed in the early development of PSO [1], which used a stochastic variable called *craziness*, but this was soon omitted from the classical PSO algorithms.

Various other solutions have been proposed for preventing premature convergence: objective functions that change over time [23]; noisy evaluation of the function objective [24]; repulsion to keep particles away from the optimum [25]; dispersion between particles that are too close to one another [26]; reduction of the attraction of the swarm centre to prevent the particles clustering too tightly in one region of the search space [27]; hybrids with other metaheuristic such as genetic algorithms [28,29]; or ant colony optimization [30], etc. For an up-to-date overview of the PSO, see Poli et al. [14].

Finally, several researchers have experimented with the swarm topology. Mendes [31] carried out an in-depth study of the effect of static topologies. Clerc [18] and Liang and Suganthan [32] conducted interesting research into dynamic topologies. In the present study we consider a static fully connected graph, which is a static topology [14].

The papers mentioned above are a sample of the wider research effort aimed at preventing premature convergence and enabling the PSO algorithm to escape from a local optimum. Although there are a lot of proposals for preventing premature convergence (e.g. by modifying the value of the inertia weight; by introducing diversity adding more particles,

passive congregation or turbulence; hybridization, etc.), only three types of feedback of the current state of the search are taken, which are used to modify some of the parameter values accordingly: (1) no feedback, because all of the parameter values are constant over the execution of the algorithm; (2) the current iteration of the algorithm and (3) the performance of the algorithm.

In the present study we propose a new method for introducing diversity into the swarm by adding a term of randomness to the particle velocity. This term of randomness is weighted by a coefficient that we call the *diversity coefficient*. We take feedback of the heterogeneity of the swarm to control the degree of diversity introduced (i.e. the value of the diversity coefficient). To our knowledge, no PSO algorithm uses a measure of the swarm heterogeneity to dynamically modify the values of its parameters. It seems reasonable to assume that the value of the diversity coefficient should not to be set before running the PSO algorithm. Instead, it is preferable to allow the value to change dynamically during the execution of the algorithm according to the convergence of the swarm (i.e. the convergence of the search): the more heterogeneous the population, the smaller the value of the diversity coefficient, and vice versa.

We use the response time variability problem (RTVP) to test our proposal. The RTVP is a scheduling problem that was recently defined in the literature [33] and is very difficult to solve optimally (it is NP-hard). This problem has a wide range of real-life applications: it occurs whenever products, clients or jobs need to be sequenced in such a way that the variability in the time between the instants at which they receive the necessary resources is minimized. For example, it can be used to regularly sequence models in the automobile industry [34], to broadcast video and audio data frames of applications over asynchronous transfer mode networks as constantly as possible [35], in the stride scheduling technique [36] and in the periodic machine maintenance problem when the distances between consecutive services of the same machine are equal [37].

García et al. [38] used the classical PSO metaheuristic to solve the RTVP and proposed four PSO variations. The best of these, called *PSO-MIF* by the authors, is the best heuristic method that has been published to date for solving non-small instances of the RTVP.

In order to compare our PSO algorithm (that we call *PSO-c₃dyn*), we have adapted a representative set of PSO algorithms for solving the RTVP: the PSO with a velocity to the best current point [6]; the PSOPC [16]; the two DPSO [21]; the PSO with turbulence [22] and two classical PSOs: the *PSO-MIF* adaptation [38] and a constriction PSO (CPSO) [39]. Moreover, we propose some variants of these algorithms that incorporate our proposed control mechanism of diversity or our proposed random velocity.

A computational experiment is carried out and it is shown that the best results are achieved when using the control mechanism of diversity proposed in this paper.

The remainder of the paper is organized as follows. Section 2 describes the scheme of the classical PSO and our newly proposal for introducing diversity according to the convergence of the population. Section 3 presents a formal definition of the RTVP. Section 4 explains our PSO algorithm and the adaptation of the mentioned PSO algorithms to solve the RTVP. Section 5 provides the results of the computational experiment. Finally, some conclusions and suggestions for future research are given in Section 6.

2. Particle swarm optimization

First, the classical PSO is briefly explained in Section 2.1. Next, the method that we propose for introducing diversity according to the convergence of the population is presented in Section 2.2. Finally, how to set the value of the coefficient that weights the diversity introduced into the population is discussed in Section 2.3.

2.1. Classical scheme of PSO

PSO was first described by Kennedy and Eberhart [1]. It is an evolutionary metaheuristic based on the behaviour of flocks of birds when they look for food.

The population of PSO or swarm is composed of particles (birds), which have an associated multi-dimensional real point in the search space (which represents a solution) and an associated velocity (the movement of the point in the multi-dimensional real space). The velocity of a particle is typically a linear combination of three types of velocity: (1) the inertia velocity; (2) the velocity to the best point found by the particle and (3) the velocity to the best point found by the swarm. The point and the velocity of each particle i of the population are iteratively modified by the algorithm as it

```

1. Set  $t = 0$ 
2. Randomly initialize positions of all particles
3. Initialize velocities of all particles with void velocities
4. While stopping criteria is not reached do:
5.   For each particle  $i$  in the swarm:
6.     Calculate fitness: Set  $f_i$  = fitness of  $X_{t,i}$ 
7.     Update  $P_{t,i}^*$ : If  $f_i$  is better than the fitness of  $P_{t,i}^*$ , then set  $P_{t,i}^*$  to  $X_{t,i}$ 
8.   End For
9.   Update  $SP_t^*$ : Set  $Xbest_t$  = the best point of the current swarm
      If  $Xbest_t$  is better than  $SP_t^*$ , then set  $SP_t^*$  to  $Xbest_t$ 
10.  For each particle  $i$  in the swarm:
11.    Update velocity: Calculate velocity  $V_{t+1,i}$  using Equation (1b)
12.    Update point: Calculate point  $X_{t+1,i}$  using Equation (2)
13.  End For
14.  Set  $t = t + 1$ 
15. End While

```

Fig. 1. Pseudocode of the classical PSO algorithm.

looks for an optimal solution. In the original PSO developed by Kennedy and Eberhart [1], the point and the velocity of the particles are modified according to the following two equations:

$$v_{t+1,i} = v_{t,i} + c_1 \cdot (P_{t,i}^* - X_{t,i}) + c_2 \cdot (SP_t^* - X_{t,i}), \quad (1a)$$

$$X_{t+1,i} = X_{t,i} + v_{t+1,i}, \quad (2)$$

where $v_{t,i}$ is the inertia velocity of the particle i at iteration t , $X_{t,i}$ is the point of the particle i at iteration t , $P_{t,i}^*$ is the best point found by the particle i up to iteration t , SP_t^* is the best point found by the swarm up to iteration t , and c_1 and c_2 are the coefficients, called *acceleration coefficients*, that weight the relevance of the last two types of velocity.

In the original PSO it was necessary to dampen the particle dynamics and the solution proposed was to maintain the velocity within the range $[-V_{\max}, +V_{\max}]$, where $V_{\max}$ was a parameter of the original PSO.

To find an appropriate value of $V_{\max}$ according to the problem to be solved was a hard task. Thus, Shi and Eberhart [40] proposed to modify the original first PSO equation (Eq. (1a)) to the following expression:

$$v_{t+1,i} = \omega \cdot v_{t,i} + c_1 \cdot (P_{t,i}^* - X_{t,i}) + c_2 \cdot (SP_t^* - X_{t,i}), \quad (1b)$$

where ω is the coefficient (called *inertia weight*) that weights the inertia velocity. Equations (1b) and (2) are the core of the classical PSO.

At each iteration of the PSO algorithm, Equations (1b) and (2) reflect the compromise of each particle between following its own explorations, moving towards the best point it has found by itself and moving towards the best point found by the swarm. Fig. 1 shows the pseudocode of the classical PSO algorithm.

Clerc and Kennedy [39] used a series of theoretical analyses to develop a strategy of *constriction coefficients*. They proposed the CPSO, in which Eq. (1a) is modified to the following expression:

$$v_{t+1,i} = \chi \cdot (v_{t,i} + c_1 \cdot (P_{t,i}^* - X_{t,i}) + c_2 \cdot (SP_t^* - X_{t,i})). \quad (1c)$$

Although CPSO is algebraically equivalent to PSO with inertia weight, CPSO can generate higher-quality solutions than PSO with inertia weight for some of the problems studied in the literature [15].

2.2. Introducing a diversity factor into PSO

As mentioned above, Trelea [12] demonstrated that the classical PSO metaheuristic always converges for some values of its parameters. Although the convergence of the metaheuristic is a desirable property, it can converge too fast and become trapped in a local optimum, particularly when PSO deals with integer variables [2].

In Section 1 several ideas for preventing a premature convergence of the PSO algorithm have been presented, including the introduction of diversity. In this paper we propose a new way to introduce diversity into the population

by adding a new type of velocity (a random velocity) to the linear combination formulated in Eq. (1b). The modified equation is as follows:

$$v_{t+1,i} = \omega \cdot v_{t,i} + c_1 \cdot (P_{t,i}^* - X_{t,i}) + c_2 \cdot (SP_t^* - X_{t,i}) + c_3 \cdot (R_{t,i} - X_{t,i}), \quad (1d)$$

where $R_{t,i}$ is a random point generated for the particle i at iteration t and c_3 (called the *diversity coefficient*) is the coefficient that weights the relevance of the new type of velocity. Note that the random velocity introduces diversity into the movements of the particles.

The value of the diversity coefficient is important in preventing PSO from converging prematurely and leaving regions of the search space unexplored. A large value of the diversity coefficient enables the PSO algorithm to carry out a wide-ranging exploration of the search space and thereby look for new promising points in exchange for a small exploitation, and vice versa for a small value of the diversity coefficient.

2.3. Parameter tuning versus parameter control

One laborious aspect of metaheuristics is choosing the right parameter values. This is a difficult task that requires considerable effort. Eiben et al. [41] draw a distinction between two principal ways of setting parameter values: parameter tuning and parameter control. Parameter tuning refers to finding and to setting the parameter values before running the algorithm, whereas parameter control refers to using parameter values that change during the execution of the algorithm.

Population-based metaheuristics such as PSO are intrinsically dynamic and the optimal parameter values might depend on which search state the algorithm is in. Therefore, the parameter values should be modified during the execution of the algorithm according to the search state. According to Eiben et al. [41], there are two ways of doing this: by using adaptive parameter control or self-adaptive parameter control.

The parameter values in adaptive parameter control are explicitly adaptive: the changes in values are given by a heuristic rule, which takes feedback from the current search state and modifies the parameter values accordingly. The input information for the current state is usually the number of iterations of the algorithm (i.e. the current iteration of the search), the performance of operators (i.e. the progress of the search) or the diversity of the population (for more details, see Eiben et al. [41]).

In self-adaptive parameter control, the parameters of the metaheuristic are incorporated into the representation of the solution. Thus, the parameter values are implicitly adaptive because they evolve together with the solutions of the population. Self-adaptive parameter control is more common in genetic algorithms in which the parameters can be incorporated into the chromosomes, which renders them subject to evolution [42,43].

In Section 2.2, we proposed introducing diversity into the PSO population by adding a random velocity weighted by the diversity coefficient (c_3). The value of the diversity coefficient may be static (i.e. its value is obtained by parameter tuning) or dynamic (i.e. its value is obtained by adaptive parameter control). The main difficulty of using a static diversity coefficient is to find a value that is (if possible) high enough to facilitate good exploration but low enough to facilitate good exploitation. Alternatively, a dynamic diversity coefficient can be used. We propose to use a control that changes the value of the diversity coefficient according to the heterogeneity of the population: the more heterogeneous the current population, the smaller the value of the diversity coefficient, and the less heterogeneous the current population, the larger the value of the diversity coefficient. To our knowledge, no PSO algorithm presented in the literature uses an adaptive parameter control whose feedback is based on the heterogeneity of the swarm.

3. The response time variability problem (RTVP)

The RTVP occurs whenever products, clients or jobs need to be sequenced so as to minimize variability in the time between the instants at which they receive the necessary resources. This combinatorial optimization problem is easy to formulate, but Corominas et al. [33] proved that it is NP-hard and, therefore, very difficult to solve optimally.

The RTVP was recently defined in the literature and first presented by Corominas et al. [33]. The RTVP is formulated as follows. Let p be the number of products, d_i the number of units of product i ($i = 1, \dots, p$) and D the total number of units ($D = \sum_{i=1}^p d_i$). Let s be a solution of an instance in the RTVP that consists of a circular sequence of units ($s = s_1 s_2 \dots s_D$), where s_j is the unit sequenced in position j of sequence s . For all product i in which $d_i \geq 2$, let t_k^i be

the distance between the positions at which the units $k + 1$ and k of product i are found (i.e. the number of positions between the units, where the distance between two consecutive positions is considered equal to 1). As the sequence is circular, position 1 comes immediately after position D ; therefore, $t_{d_i}^i$ is the distance between the first unit of product i in a cycle and the last unit of the same product in the preceding cycle. Let $\bar{t}_i$ be the average distance between two consecutive units of product i ($\bar{t}_i = D/d_i$). For all product i in which $d_i = 1$, t_1^i is equal to $\bar{t}_i$. The objective is to minimize the $RTV = \sum_{i=1}^p \sum_{k=1}^{d_i} (t_k^i - \bar{t}_i)^2$.

For example, let $p=3$, $d_A=2$, $d_B=2$ and $d_C=4$; thus, $D=8$, $\bar{t}_A=4$, $\bar{t}_B=4$ and $\bar{t}_C=2$. Any sequence is a feasible solution. For example, the sequence (C, A, C, B, C, B, A, C) is a solution, where $RTV = ((5-4)^2 + (3-4)^2) + ((2-4)^2 + (6-4)^2) + ((2-2)^2 + (2-2)^2 + (3-2)^2 + (1-2)^2) = 2 + 8 + 2 = 12$.

There are only three published works that make reference to the RTVP. Corominas et al. [33] presented the RTVP and proposed a mixed integer linear programming (MILP) model and five heuristic algorithms to solve it. Corominas et al. [44] presented an improved MILP model with a practical limit for obtaining optimal solutions of around 40 units to be scheduled. García et al. [38] presented six metaheuristic algorithms: a multi-start algorithm, a greedy randomized adaptive search procedure (GRASP) algorithm and four PSO algorithm variations. In order to solve non-small instances of the RTVP, one of the PSO algorithm variations (referred to as *PSO-MIF*) is the method published to date that obtains, on average, the best results.

4. PSO algorithms for solving the RTVP

We propose 11 different PSO algorithms for solving the RTVP. The first approach, *PSO-MIF*, is taken from García et al. [38] and is based on the classical PSO. The second algorithm, *CPSO*, is an adaptation of the CPSO proposed by Clerc and Kennedy [39]. The third algorithm, *PSO-c₃dyn*, is our proposal in which the random velocity is introduced together with the dynamic control mechanism of the value of the diversity weight according to the heterogeneity of the swarm. The fourth algorithm, *PSOCB*, incorporates the idea of replacing the velocity to the best point found with the swarm by the velocity to the current best point of the swarm [13]. The fifth algorithm, *PSOPC*, is an adaptation of the PSO with passive congregation proposed by He et al. [16]. The sixth and seventh algorithms, *DPSOvel* and *DPSOpoi*, are adaptations of the two DPSOs proposed by Xie et al. [21] in which the velocity or the point is randomly modified. The eighth algorithm, *PSO-c₃stat*, is an adaptation of the PSO with turbulence proposed by Fieldsend and Singh [22]. Finally, the 9th, 10th and 11th algorithm, *PSO-c₃dyn'*, *PSOPC'* and *DPSOpoi-c_pdyn*, are variations that we developed by merging ideas incorporated into the designs of the previous algorithms.

In the following 11 sections, each PSO algorithm is explained. In Section 4.12 we explain how the parameters of the 11 PSO algorithms were fine-tuned.

4.1. A classical PSO algorithm (*PSO-MIF*)

PSO was originally designed for working in multi-dimensional real spaces. The representation of a solution of the RTVP (and many other combinatorial optimization problems) consists of an ordered sequence of integer numbers. Therefore, the PSO metaheuristic has to be adapted to work with this type of solution representation (i.e. the point of a particle is now a sequence of integer numbers). This is done by redefining the elements (point and velocity) and the operations (external multiplication of a coefficient by a velocity, sum of velocities and sum of a velocity plus a point) of Eqs. (1b) and (2). Moreover, it is also necessary to determinate how the initial population is set and the stopping criteria

García et al. [38] proposed four variations based on the classical PSO with inertia weight [40] for solving the RTVP. The differences between these variations derive from the way in which the velocity is redefined. The PSO algorithm that produces the best results is *PSO-MIF*. In the following sections, the elements and the operations of *PSO-MIF* are briefly explained (for a more detailed explanation, see García et al. [38]).

The definitions of elements and operations are the same for all 11 PSO algorithms.

4.1.1. Point of the particle

A point consists of a solution represented by a D -length array that contains the sequence of D units to be scheduled. For example, a point could be the sequence (C, A, C, B, C, B, A, C) .

4.1.2. Velocity of the particle

The expression $(X_2 - X_1)$, where X_2 and X_1 are two points, represents the difference between two points and the velocity needed to go from X_1 to X_2 . This velocity is an ordered list of transformations (called movements) that must be applied sequentially to the particle so that its current point, X_1 , changes to the other one, X_2 . A movement is a pair of values (α/j) . For each position u in the sequence (point) X_1 , the algorithm determines whether the unit that is in position u of sequence X_1 is the same unit that is in position u of sequence X_2 . If the units are different, α is the unit in position u of X_2 and j is equal to position u . Thus, this movement denotes that to go from the sequence X_1 to the sequence X_2 , the unit in position j must be exchanged for the unit α .

For example, let $X_2 = (A_1, C_1, B_2, C_2, A_2, C_4, B_1, C_3)$ and $X_1 = (A_1, B_1, C_2, C_1, B_2, C_4, A_2, C_3)$. The sub-indices of the units are fictitious identifiers used to distinguish between the units of the same product. Thus, the velocity $(X_2 - X_1)$ is formed by the list of movements $[(C_1/2), (B_2/3), (C_2/4), (A_2/5), (B_1/7)]$, which are the movements for moving $X_1 - X_2$:

$$\begin{aligned} X_1 &= (A_1, B_1, C_2, C_1, B_2, C_4, A_2, C_3), \\ (C_1/2) &\rightarrow (A_1, \mathbf{C}_1, C_2, \mathbf{B}_1, B_2, C_4, A_2, C_3), \\ (B_2/3) &\rightarrow (A_1, C_1, \mathbf{B}_2, B_1, \mathbf{C}_2, C_4, A_2, C_3), \\ (C_2/4) &\rightarrow (A_1, C_1, B_2, \mathbf{C}_2, \mathbf{B}_1, C_4, A_2, C_3), \\ (A_2/5) &\rightarrow (A_1, C_1, B_2, C_2, \mathbf{A}_2, C_4, \mathbf{B}_1, C_3), \\ (B_1/7) &\rightarrow (A_1, C_1, B_2, C_2, A_2, C_4, \mathbf{B}_1, C_3) = X_2. \end{aligned}$$

4.1.3. External multiplication of a coefficient by a velocity

The values of the coefficients ω , c_1 and c_2 in Eq. (1b) are between 0 and 1. When a coefficient is multiplied by a velocity, it indicates the probability of each movement to be applied. For example, if we multiply the coefficient 0.6 by the velocity $[(C_1/2), (B_2/3), (C_2/4), (A_2/5), (B_1/7)]$, five random numbers between 0 and 1 are generated for comparison with the value 0.6. If the random number is lower than 0.6, the movement is applied. Therefore, if the values of the random numbers are 0.8, 0.3, 0.7, 0.4 and 0.2, movements $(B_2/3)$, $(A_2/5)$ and $(B_1/7)$ are applied, whereas movements $(C_1/2)$ and $(C_2/4)$ are not. The resulting velocity of the multiplication is therefore $[(B_2/3), (A_2/5), (B_1/7)]$, which, as previously stated, represents a list of movements to be applied to a point.

4.1.4. Sum of velocities

The sum of two velocities is simply the concatenation of their own list of movements.

4.1.5. Sum of a velocity plus a point

The sum of a velocity plus a point gives the result of sequentially applying each movement of the velocity to the point.

4.1.6. Initial population

The initial population is generated by setting a void velocity and a random point for each particle. As has been previously mentioned, each point consists of a solution represented by a sequence of integer numbers. A random solution is generated as follows: for each position in the sequence, a product to be sequenced is chosen at random. The probability of each product is equal to the number of units of this product that remain to be sequenced divided by the total number of units that remain to be sequenced.

4.1.7. Stopping criteria

The PSO algorithm stops after it has run for a preset time.

4.2. A CPSO algorithm

As explained in Section 2.1, Clerc and Kennedy [39] proposed a PSO algorithm based on *constriction coefficients*, whose core is Eqs. (1c) and (2). We propose a Constriction PSO algorithm (called CPSO) for solving the RTVP in

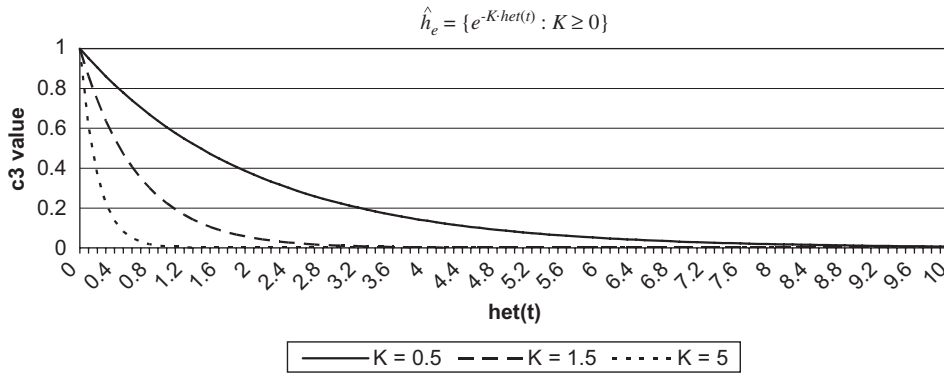


Fig. 2. Examples of heuristic rules from $\hat{h}_e$ for obtaining the value of the diversity coefficient (c_3).

which the particles behave according to Eqs. (1c) and (2) and the parameter values (the size of the swarm and the coefficients χ , c_1 and c_2) are preset.

4.3. A PSO algorithm with a dynamic value of the diversity coefficient (PSO- c_3 dyn)

As explained in Section 2.2, the diversity coefficient (parameter c_3 in Eq. (1d)) weights the relevance of the random velocity. This random velocity is important in preventing PSO from converging prematurely and leaving regions of the search space unexplored. A large value of the diversity coefficient enables to PSO algorithm to carry out a wide-ranging exploration of the search space and thereby look for new promising points in exchange for a small exploitation, and vice versa for a small value of the diversity coefficient. Therefore, we propose a new PSO algorithm (called *PSO- c_3 dyn*) in which an adaptive parameter control dynamically changes the diversity coefficient.

A good heuristic rule for dynamically changing the value of the diversity coefficient should be one in which the more heterogeneous the current population, the smaller the value given to the diversity coefficient and vice versa. First, a measure of heterogeneity of the population at iteration t is needed. The following expression is used:

$$\text{het}(t) = \frac{\sum_{i \in P} |v_{t,i}|}{D \cdot |P|}, \quad (3)$$

where P is the population, $|v_{t,i}|$ is the number of movements of the velocity of particle i at iteration t , D is the number of units to be sequenced and $|P|$ is the size of the population P . As the diversity is introduced into the movements of the particles, the measure of heterogeneity $\text{het}(t)$ records the movement of the particles.

Given a specific value of the population heterogeneity ($\text{het}(t)$) at iteration t , it is necessary to set the corresponding value of the diversity coefficient (c_3). There are infinite possible heuristic rules for obtaining the value of the diversity coefficient according to $\text{het}(t)$. In the present study we used a set of rules according to which the value of the diversity coefficient decreases exponentially with $\text{het}(t)$ (let the set be called $\hat{h}_e$). Set $\hat{h}_e$ is formulated as $\hat{h}_e = \{e^{-K \cdot \text{het}(t)} : K \geq 0\}$. Fig. 2 shows some heuristic rules of $\hat{h}_e$ in graph form.

According to the value of K , a heuristic rule from set $\hat{h}_e$ is used. Therefore, K is considered to be one of the parameters of *PSO- c_3 dyn* in addition to the population size and the coefficients ω , c_1 and c_2 that need to be fine-tuned.

4.4. A PSO algorithm with a current best swarm point (PSOCB)

Clerc [13] suggested a possible classical PSO variant in which the velocity to the best point found by the swarm is replaced by the velocity to the best current point. Thus, Eq. (1b) is replaced by the following equation:

$$v_{t+1,i} = \omega \cdot v_{t,i} + c_1 \cdot (P_{t,i}^* - X_{t,i}) + c_2 \cdot (SCP_t^* - X_{t,i}), \quad (1e)$$

where SCP_t^* is the best point of the swarm at iteration t .

We propose a discrete version (that we call *PSOCB*) in which the parameter values (the size of the swarm and the coefficients ω , c_1 and c_2) are preset.

4.5. A PSO algorithm with passive congregation (PSOPC)

He et al. [16] developed a PSO called PSOPC, which models passive congregation behaviour. In PSOPC, Eq. (1b) is changed to the following equation:

$$v_{t+1,i} = \omega \cdot v_{t,i} + c_1 \cdot (P_{t,i}^* - X_{t,i}) + c_2 \cdot (SP_t^* - X_{t,i}) + c_3 \cdot (P_{t,i}^R - X_{t,i}), \quad (1f)$$

where $P_{t,i}^R$ is the point of a particle selected at random for the swarm for the particle i at iteration t . Note that the fourth terms of Eq. (1d) (*PSO-c3dyn*) and Eq. (1f) (*PSOPC*) are different: Eq. (1f) uses a point of an existing particle ($P_{t,i}^R$), whereas Eq. (1d) uses a randomly generated point ($R_{t,i}$).

He et al. [16] used an adaptive parameter control according to the current iteration of the algorithm to dynamically set the values of the coefficients ω and c_3 : ω decreases linearly and c_3 increases linearly when the number of iterations increases. We adopt the same strategy in the *PSOPC* algorithm.

Thus, the parameter values that need to be preset are the size of the swarm, the coefficients c_1 and c_2 , the starting values of ω and c_3 and their decreasing and increasing slopes.

4.6. A PSO algorithm with dissipative velocity (DPSOvel)

Xie et al. [21] added the following equation to the classical PSO algorithm, which is applied after Eqs. (1b) and (2) to introduce additional diversity into the velocity of the particles:

$$\text{IF } (\text{rand}() < c_v) \text{ THEN } v_{t,i}^d = \text{rand}() \cdot v_{\max,d}, \quad (3a)$$

where c_v is a factor between 0 and 1, $v_{t,i}^d$ is the component d of the velocity $v_{t,i}$, $\text{rand}()$ is a random value between 0 and 1 and $v_{\max,d}$ is an upper bound of the component d of the velocity.

This PSO algorithm was developed to solve continuous optimization problems; hence, the way of introducing additional diversity has to be adapted to our discrete PSO algorithm (*DPSOvel*). Each movement of a velocity (which is a list of movements) has a probability c_v of been swapped with another, randomly selected movement.

The parameter values (the size of the swarm, the coefficients ω , c_1 and c_2 and the factor c_v) are preset.

4.7. A PSO algorithm with dissipative point (DPSOpoi)

Xie et al. [21] added the following equation to the classical PSO algorithm, which is applied after Eqs. (1b) and (2) to introduce additional diversity into the point of the particles:

$$\text{IF } (\text{rand}() < c_p) \text{ THEN } x_{t,i}^d = \text{Random}(l_d, u_d), \quad (3b)$$

where c_p is a factor between 0 and 1, $x_{t,i}^d$ is the component d of the point $X_{t,i}$, $\text{Random}(l_d, u_d)$ is a random value between l_d and u_d , l_d is a lower bound of the component d of the point and u_d is an upper bound of the component d of the point.

This PSO algorithm was developed to solve continuous optimization problems; hence, the way of introducing additional diversity has to be adapted to our discrete PSO algorithm (*DPSOpoi*). For each position of the point (which is a sequence), the position has a probability c_p of being swapped with another, randomly selected position.

The parameters values (the size of the swarm, the coefficients ω , c_1 and c_2 and the factor c_p) are preset.

4.8. A PSO algorithm with turbulence (PSO-c3stat)

Fieldsend and Singh [22] introduced a random velocity (that they call *turbulence*) into the classical PSO for solving continuous optimization problems. In this PSO algorithm, Eq. (1b) is replaced by the following equation:

$$v_{t+1,i} = \omega \cdot v_{t,i} + c_1 \cdot (P_{t,i}^* - X_{t,i}) + c_2 \cdot (SP_t^* - X_{t,i}) + c_3 \cdot RV_{t,i}, \quad (1g)$$

where $RV_{t,i}$ is a random velocity generated for the particle i at iteration t .

In our adaptation we propose to generate the random velocity so that the particle moves to a randomly generated point, i.e. our proposal to introduce diversity into the swarm (see Section 2.2). The value of c_3 is static in the Fieldsend and Singh algorithm, so we call our adaptation *PSO- c_3stat* .

The parameters values of *PSO- c_3stat* (the size of the swarm, the coefficients ω , c_1 , c_2 and c_3) are preset.

4.9. A *PSO- c_3dyn* variant (*PSO- c_3dyn'*)

We propose to vary *PSO- c_3dyn* (see Section 4.3) by introducing diversity with passive congregation (i.e. as in *PSOPC* (see Section 4.5)) instead of random velocity. We call this variant *PSO- c_3dyn'* .

4.10. A *PSOPC* variant (*PSOPC'*)

We propose to vary *PSOPC* (see Section 4.5) by introducing diversity with random velocity (i.e. as in *PSO- c_3dyn* (see Section 4.3)) instead of passive congregation. We call this variant *PSOPC'*.

4.11. A *DPSOpoi* variant (*DPSOpoi- c_pdyn*)

We propose to vary *DPSOpoi* (see Section 4.7) by using our adaptive control based on the population heterogeneity to dynamically change the value of the parameter c_p (i.e. the diversity control mechanism used in *PSO- c_3dyn* (see Section 4.3)).

4.12. Fine-tuning of the PSO parameters

Fine-tuning the parameters of a metaheuristic algorithm is almost always a difficult task. Although the parameter values are extremely important because the results of the metaheuristic for each problem are very sensitive to them, the selection of parameter values is commonly justified in one of the following ways [41,45]: (1) “by hand” on the basis of a small number of experiments that are not specifically referenced; (2) by using the general values recommended for a wide range of problems; (3) by using the values reported to be effective in other similar problems or (4) by choosing values without any explanation.

Adenso-Díaz and Laguna [45] proposed a new technique called CALIBRA for fine-tuning the parameters of heuristic and metaheuristic algorithms. CALIBRA is based on Taguchi’s fractional factorial experimental designs coupled with a local search procedure. CALIBRA has been chosen for fine-tuning the PSO parameters using a set of 60 representative training instances (generated as explained in Section 5).

García et al. [38] used CALIBRA to fine-tune *PSO-MIF*. The same technique is also used to fine-tune the other proposed PSO algorithms. The following parameter values are obtained:

1. *PSO-MIF*: size of the swarm = 21, $\omega = 0.87$, $c_1 = 0.87$ and $c_2 = 0.37$;
2. *CPSO*: size of the swarm = 75, $\chi = 0.75$, $c_1 = 0.75$ and $c_2 = 0.75$;
3. *PSO- c_3dyn* : size of the swarm = 13, $\omega = 0.87$, $c_1 = 0.87$, $c_2 = 0.87$ and $K = 3.80$;
4. *PSOCB*: size of the swarm = 25, $\omega = 0.87$, $c_1 = 0.75$ and $c_2 = 0.37$;
5. *PSOPC*: size of the swarm = 25, $\omega = 0.63$, $c_1 = 0.37$, $c_2 = 0.75$, $c_3 = 0.63$, ω slope = -0.00250 and c_3 slope = 0.00437 ;
6. *DPSOvel*: size of the swarm = 25, $\omega = 0.37$, $c_1 = 0.87$, $c_2 = 0.25$ and $c_v = 0.087$;
7. *DPSOpoi*: size of the swarm = 25, $\omega = 0.25$, $c_1 = 0.75$, $c_2 = 0.25$ and $c_p = 0.050$;
8. *PSO- c_3stat* : size of the swarm = 8, $\omega = 0.13$, $c_1 = 0.75$, $c_2 = 0.87$ and $c_3 = 0.25$;
9. *PSO- c_3dyn'* : size of the swarm = 13, $\omega = 0.87$, $c_1 = 0.25$, $c_2 = 0.87$ and $K = 6.30$;
10. *PSOPC'*: size of the swarm = 87, $\omega = 0.87$, $c_1 = 0.87$, $c_2 = 0.87$, $c_3 = 0.25$, ω slope = -0.00625 and c_3 slope = 0.00125 ;
11. *DPSOpoi- c_pdyn* : size of the swarm = 13, $\omega = 0.75$, $c_1 = 0.13$, $c_2 = 0.75$ and $K = 8.70$.

Since CALIBRA cannot fine-tune more than five parameters, *PSOPC* and *PSOPC'* (which have seven parameters) are fine-tuned in two steps. In the first step, the initial values of ω and c_3 are set to 0.9 and 0.4, respectively (as in He et al. [16]), and the remaining parameters (the size of the swarm, c_1 , c_2 , the ω slope and the c_3 slope) are fine-tuned.

In the second step, the values of c_1 and c_2 are set at the values obtained in the first step and the remaining parameters (the size of the swarm, ω , c_3 , the ω slope and the c_3 slope) are fine-tuned.

5. Computational experiment

The computational experiment for the 11 proposed PSO algorithms was carried out for the same instances and conditions used in García et al. [38]. That is, the algorithms ran 740 instances that were grouped into four classes (185 instances in each class) according to their size. The instances in the first class (*CAT1*) were generated using a random value of D (number of units) uniformly distributed between 25 and 50 and a random value of p (number of products) uniformly distributed between 3 and 15; for the second class (*CAT2*), D was between 50 and 100 and p between 3 and 30; for the third class (*CAT3*), D was between 100 and 200 and p between 3 and 65 and for the fourth class (*CAT4*), D was between 200 and 500 and p between 3 and 150. For all instances and for each type of product $i = 1, \dots, p$, a random value of d_i (number of units of the product i) was between 1 and $\lfloor (D - p + 1)/2.5 \rfloor$ such that $\sum_{i=1}^p d_i = D$. All algorithms were coded in Java and the computational experiment was carried out using a 3.4 GHz Pentium IV with 512 Mb of RAM.

The 11 algorithms were run for 50 s for each instance. Table 1 shows the averages of the RTV values to be minimized for the global of 740 instances and for each class of instances (*CAT1*–*CAT4*) obtained with the PSO algorithms.

Table 1 shows that for the global of all instances, the results of *PSO-MIF*, which was until now the best method proposed in the literature for solving the RTVP, are improved on by the results of three PSO algorithms: *DPSOpoi-c_pdyn* is 45.60% better, *PSO-c₃dyn* is 17.84% better and *PSOCB* is 2.19% better. There are three algorithms that obtain slightly poorer results than *PSO-MIF* (*DPSOvel*, *CPSO* and *DPSOpoi*) and the four remaining algorithms provide much poorer results (*PSO-c₃dyn'*, *PSOPC*, *PSO-c₃stat* and *PSOPC'*). If we consider the results according to class, the three best algorithms for *CAT1* instances are *PSO-c₃dyn*, *DPSOpoi-c_pdyn* and *DPSOpoi* (76.34, 75.29 and 74.20% better than *PSO-MIF*, respectively); the three best algorithms for *CAT2* instances are *DPSOpoi*, *DPSOpoi-c_pdyn* and *PSO-c₃dyn* (88.11, 87.91 and 86.55% better than *PSO-MIF*, respectively); the three best algorithms for *CAT3* instances are *DPSOpoi-c_pdyn*, *DPSOpoi* and *PSO-c₃dyn* (79.66, 72.98 and 57.95% better than *PSO-MIF*, respectively) and the three best algorithms for *CAT4* instances are *DPSOpoi-c_pdyn*, *PSO-c₃dyn* and *PSOCB* (41.60, 12.81 and 2.84% better than *PSO-MIF*, respectively).

The results in Table 1 show that our proposed mechanism for controlling the degree of diversity introduced into the swarm is very effective. The best and second best results are obtained with two PSO algorithms that apply our control mechanism of diversity (*DPSOpoi-c_pdyn* and *PSO-c₃dyn*, respectively). *DPSOpoi-c_pdyn* produces an improvement of 47.39% with respect to *DPSOpoi*, which is its static version, i.e. the degree of diversity introduced into the swarm is preset and is not changed by the proposed control mechanism. In addition, *PSO-c₃dyn* produces an improvement of 62.66% with respect to *PSO-c₃stat*, which is its static version. Our proposal of introducing diversity through a random velocity works well when the degree of diversity is dynamically controlled according to our proposal.

In Table 2 we compare the number of times that each PSO algorithm reaches the best RTV value obtained with all 11 algorithms. The results are shown for the 740 instances overall and for each class of instances.

Table 1
Averages of the RTV values for 50 s

	Global	<i>CAT1</i>	<i>CAT2</i>	<i>CAT3</i>	<i>CAT4</i>
<i>DPSOpoi-c_pdyn</i>	4625.54	16.42	51.34	610.34	17,824.04
<i>PSO-c₃dyn</i>	6986.05	15.72	57.10	1261.81	26,609.56
<i>PSOCB</i>	8316.51	73.79	433.98	3106.96	29,651.33
<i>PSO-MIF</i>	8502.83	66.44	424.60	3000.52	30,519.76
<i>DPSOvel</i>	8686.47	19.28	179.60	2287.05	32,259.96
<i>CPSO</i>	8774.06	74.51	478.13	3478.72	31,064.89
<i>DPSOpoi</i>	8792.70	17.14	50.50	810.58	34,292.58
<i>PSO-c₃dyn'</i>	11,133.09	146.77	804.12	5251.08	38,330.39
<i>PSOPC</i>	14,579.82	82.03	563.05	4021.67	53,652.54
<i>PSO-c₃stat</i>	18,707.12	40.41	853.26	7959.23	65,975.58
<i>PSOPC'</i>	19,626.03	145.26	1178.29	9086.24	68,094.33

Table 2
Number of times that the best solution is reached

	Global	CAT1	CAT2	CAT3	CAT4
<i>DPSOpoi-c_pdyn</i>	468	71	70	145	182
<i>PSO-c₃dyn</i>	142	88	52	1	1
<i>PSOCB</i>	1	0	0	0	1
<i>PSO-MIF</i>	0	0	0	0	0
<i>DPSOvel</i>	60	50	9	0	1
<i>CPSO</i>	1	1	0	0	0
<i>DPSOpoi</i>	179	66	74	39	0
<i>PSO-c₃dyn'</i>	0	0	0	0	0
<i>PSOPC</i>	0	0	0	0	0
<i>PSO-c₃stat</i>	46	46	0	0	0
<i>PSOPC'</i>	0	0	0	0	0

Table 3
Average σ dispersion with respect to the best solution found

	Global	CAT1	CAT2	CAT3	CAT4	* ^a
<i>DPSOpoi-c_pdyn</i>	0.11	0.29	0.16	0.01	0.00	(1)
<i>PSO-c₃dyn</i>	0.64	0.19	0.23	1.62	0.51	(3)
<i>PSOCB</i>	72.20	44.44	159.40	83.77	1.21	(6)
<i>PSO-MIF</i>	65.63	35.68	150.51	75.15	1.17	(5)
<i>DPSOvel</i>	10.87	0.97	18.44	22.96	1.10	(4)
<i>CPSO</i>	81.87	42.07	190.13	93.45	1.83	(7)
<i>DPSOpoi</i>	0.41	0.41	0.10	0.18	0.95	(2)
<i>PSO-c₃dyn'</i>	313.65	261.54	692.52	295.26	5.30	(10)
<i>PSOPC</i>	123.10	52.95	292.21	141.44	5.83	(8)
<i>PSO-c₃stat</i>	302.44	8.73	613.62	566.83	20.58	(9)
<i>PSOPC'</i>	622.75	199.92	1394.34	874.23	22.52	(11)

^aOrder according to the average σ dispersion for the global of all instances.

As we expect from the results in Table 1, Table 2 shows that *DPSOpoi-c_pdyn* reaches the best solution the greatest number of times (in 63.24% for the global of all instances). Surprisingly, although *DPSOpoi* is the seventh best algorithm according to the obtained RTV values, it is the algorithm that reaches the best solution the second-highest number of times (in 24.19% for the global of all instances) followed by *PSO-c₃dyn* (in 19.19% for the global of all instances). In addition, *DPSOvel* and *PSO-c₃stat* reach the best solution the fourth- and the fifth-highest number of times, respectively.

To complete the analysis of the results, their dispersion is observed. We use the following expression to define a measure of the dispersion (let it be called σ) of the RTV values obtained by each algorithm, *alg*, for a given instance, *ins*:

$$\sigma(\text{alg}, \text{ins}) = \left(\frac{\text{RTV}_{\text{ins}}^{(\text{alg})} - \text{RTV}_{\text{ins}}^{(\text{best})}}{\text{RTV}_{\text{ins}}^{(\text{best})}} \right)^2, \quad (4)$$

where $\text{RTV}_{\text{ins}}^{(\text{alg})}$ is the RTV value of the solution obtained with the algorithm *alg* for the instance *ins* and $\text{RTV}_{\text{ins}}^{(\text{best})}$ is the best RTV value of the solutions obtained with the 11 algorithms for the instance *ins*. Table 3 shows the average σ dispersion for the 740 overall instances and for each class of instances.

We can see from Table 3 that *DPSOpoi-c_pdyn*, *DPSOpoi* and *PSO-c₃dyn* have the lowest average σ dispersion for the global of all instances. Therefore, the two algorithms that apply the proposed mechanism for controlling the diversity introduced (*DPSOpoi-c_pdyn* and *PSO-c₃dyn*) not only produce the best RTV values but also exhibit very stable behaviour, i.e. when they do not obtain the best result for a given instance, they obtain a value that is very close to it. On the other hand, although *DPSOpoi* and *DPSOvel* produce slightly worse RTV values than *PSOCB* and *PSO-MIF*, they exhibit much more stable behaviour.

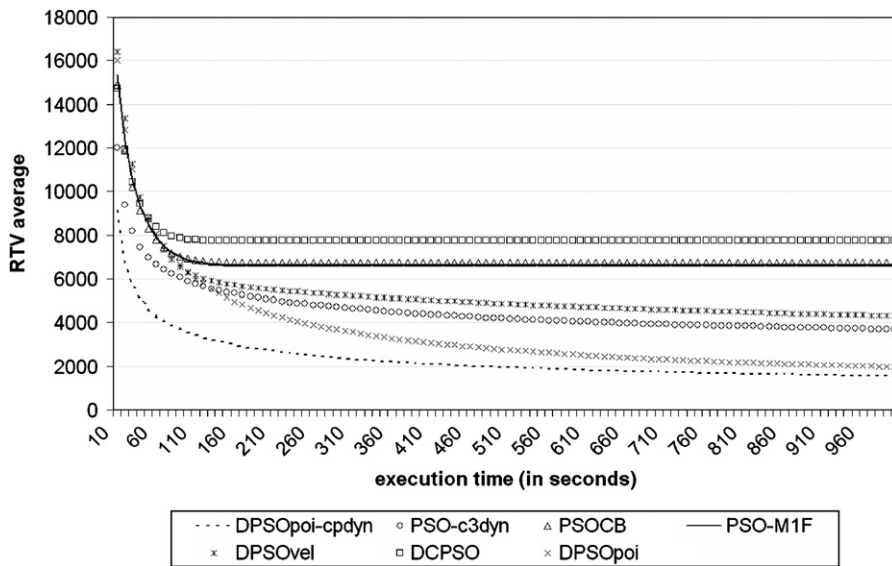


Fig. 3. Average of the RTV values obtained over the computing time.

Table 4
Averages of the RTV obtained values for 1000 s

	Global	CAT1	CAT2	CAT3	CAT4
<i>DPSOpoi-c_pdyn</i>	1537.34	14.35	46.55	143.95	5944.51
<i>DPSOpoi</i>	1980.20	14.63	46.13	142.58	7717.47
<i>PSO-c₃dyn</i>	3696.44	13.83	42.18	391.54	14,338.20
<i>DPSOvel</i>	4312.30	17.75	84.17	1036.87	16,110.42
<i>PSO-M1F</i>	6619.34	66.44	424.54	3000.51	22,985.85
<i>PSOCB</i>	6731.24	73.79	433.98	3106.96	23,310.24
<i>CPSO</i>	7746.85	74.51	478.13	3478.72	26,956.02
<i>PSOPC</i>	8838.70	82.03	563.05	4021.67	30,688.03
<i>PSO-c₃dyn'</i>	11,133.09	146.77	804.12	5251.08	38,330.39
<i>PSO-c₃stat</i>	16,212.08	16.75	592.64	6520.72	57,718.22
<i>PSOPC'</i>	18,495.01	138.76	1056.59	8414.15	64,370.53

The results in Tables 1 and 2 show that the main differences between the seven best PSO algorithms (*DPSOpoi-c_pdyn*, *PSO-c₃dyn*, *PSOCB*, *PSO-M1F*, *DPSOvel*, *CPSO* and *DPSOpoi*) are in the average values of the CAT1, CAT2 and CAT3 instances, whereas the average values of the CAT4 instances are more similar (except for those obtained by *DPSOpoi-c_pdyn*). This may occur because, on average, some PSO algorithms may converge to a local minimum on 50 s for small and medium instances (CAT1, CAT2 and CAT3 instances), whereas other algorithms continue to explore the search space. However, 50 s might not be enough time for the PSO algorithms to converge for large instances (CAT4 instances). Fig. 3 shows how the averages of the RTV values for the global of all instances decrease over the computing time for the seven best algorithms.

Fig. 3 shows that the RTV values of the seven algorithms decrease exponentially during the first 110 s of computing time. At this point, the three algorithms into which no diversity is introduced (*PSOCB*, *PSO-M1F* and *CPSO*) have converged for almost all the instances and remain trapped in a local optimum during the remaining computing time. In contrast, the other four algorithms (*DPSOpoi-c_pdyn*, *PSO-c₃dyn*, *DPSOvel* and *DPSOpoi*) introduce diversity and are able to move away from the local optimum. Therefore, these four algorithms continue to explore new regions of the search space and find better local optima during the remaining computing time.

Table 4 shows the averages of the RTV values for the global of all instances and for each class of instances (CAT1–CAT4) obtained with the 11 PSO algorithms when they are run for 1000 s.

Table 4 shows that *DPSOpoi-c_pdyn* also produces the best average for the global of all instances when the algorithms are run for 1000 s. However, the second best algorithm is now *DPSOpoi*, followed by *PSO-c₃dyn*. We can see from Fig. 3 that the results obtained by *PSO-c₃dyn* are better than those obtained by *DPSOpoi* up to an execution time of 130 s. Thus, it is recommended to use *PSO-c₃dyn* instead of *DPSOpoi* for execution times of less than 2 min. Anyway, Fig. 3 shows that it is always advisable to use *DPSOpoi-c_pdyn*, in which the way of introducing diversity into the swarm is based on one proposal by Xie et al. [21] and the degree of diversity is regulated by the control mechanism that we propose in this paper.

There are four algorithms that improve on the results obtained by *PSO-MIF* when the total computation time equals 1000 s: *DPSOpoi-c_pdyn* (76.78% better), *DPSOpoi* (70.08% better), *PSO-c₃dyn* (44.16% better) and *DPSOvel* (34.85% better). On the other hand, the poor results shown in Tables 1 and 3 for *PSOPC* and *PSO-c₃dyn'* show that proposal of using passive congregation to introduce diversity into the swarm for a discrete optimization problem such as the RTVP is unsuccessful.

6. Conclusions and future research

PSO is an evolutionary metaheuristic proposed by Kennedy and Eberhart [1], which has achieved good results for several types of problems. The metaheuristic uses an analogy of the flocking behaviour of birds to look for the optimal solution. PSO has the desirable property of convergence, but this convergence can be premature, in which case certain regions of the search space remain unexplored. To prevent this, several strategies have been proposed in the literature, some of which are based on introducing diversity into the swarm. Most published PSO applications are designed to solve continuous optimization problems, but there are few PSO applications for solving discrete optimization problems.

We propose a PSO algorithm (which we call *PSO-c₃dyn*) for solving a discrete optimization problem. To develop *PSO-c₃dyn*, two novel ideas are introduced: (1) a new way of introducing diversity into the swarm based on introducing a random velocity and (2) a mechanism for controlling the degree of diversity introduced into the swarm that takes feedback from the heterogeneity of the swarm.

The algorithm is designed to solve the RTVP, which is a scheduling problem that occurs whenever products, clients or jobs need to be sequenced in such a way that the variability in the time between the instants at which they receive the necessary resources is minimized. The *PSO-MIF* algorithm proposed by García et al. [38] is currently the best published algorithm for solving non-small instances of the RTVP.

We propose nine more PSO algorithms (in addition to *PSO-MIF*) for comparison with *PSO-c₃dyn*, which are used to test the way of introducing diversity and the control mechanism of the diversity that we propose. Eight of these nine algorithms are based on PSO algorithms that have been tested in continuous optimization problems but not in discrete optimization problems. Therefore, this paper tests the validity of these algorithms when applied to discrete optimization problems such as the RTVP. In addition, we demonstrate that CALIBRA is very useful for fine-tuning the parameters of the PSO algorithms.

The results of the computational experiment show that the proposed mechanism for controlling the degree of diversity introduced into the swarm works very well. The best results obtained, on average, are obtained under this diversity control. Thus, this paper presents a control mechanism that is very effective and contributes to improve the performance of PSO. Moreover, in this paper the previous results published in the literature of the RTVP are improved.

The two novel ideas proposed in our paper (random velocity and diversity control) are very easy to adapt to a continuous space. Therefore, our future research will focus on testing the proposed diversity control in PSO algorithms for solving continuous optimization problems and determining whether it improves the overall results. Since the proposed way of introducing diversity by using a random velocity works correctly, we will also test it in continuous optimization problems.

Acknowledgements

The authors are very grateful to Professor Albert Corominas (Technical University of Catalonia) and to the anonymous reviews for his valuable comments, which have helped to enhance this paper. The authors gratefully acknowledge the support of grant DPI2007-61905 (Ministerio de Educación y Ciencia, Spain, and FEDER).

References

- [1] Kennedy J, Eberhart RC. Particle swarm optimization. In: IEEE international conference on neural networks, Australia; 1995.
- [2] Hu X, Eberhart RC, Shi Y. Recent advances in particle swarm. In: IEEE congress on evolutionary computation, Portland, Oregon, USA; 2004.
- [3] Chau KW. Particle swarm optimization training algorithm for ANNs in stage prediction of Shing Mun River. *Journal of Hydrology* 2006;329: 363–7.
- [4] Geethanjali M, Mary Raja Slochanal S, Bhavani R. PSO trained ANN-based differential protection scheme for power transformers. *Neurocomputing*, 2007, in press (corrected proof).
- [5] Andrés C, Pastor R, Framiñán JM. Optimización mediante cúmulo de partículas del problema de secuenciación CONWIP. In: Eighteenth conference on statistics and operations research SEIO'04, Cádiz, Spain; 2004.
- [6] Liao CJ, Tseng CT, Luarn P. A discrete version of particle swarm optimization for flowshop scheduling problems. *Computer & Operations Research* 2007;34:3099–111.
- [7] Secrest B. Travelling salesman problem for surveillance mission using PSO. PhD thesis, Air Force Institute of Technology, Ohio, USA; 2001.
- [8] Tasgetiren MF, Liang Y-C, Sevkli M, Gencyilmaz G. A particle swarm optimization algorithm for makespan and total flowtime minimization in permutation flowshop sequencing problem. *European Journal of Operational Research* 2007;177:1930–47.
- [9] Hu X, Eberhart RC. Multiobjective optimization using dynamic neighborhood particle swarm optimization. In: IEEE congress on evolutionary computation, Honolulu, USA; 2002.
- [10] Yin P-Y, Yu S-S, Wang P-P, Wang Y-T. Multi-objective task allocation in distributed computing systems by hybrid particle swarm optimization. *Applied Mathematics and Computation* 2007;184:407–20.
- [11] Pan Q-K, Tasgetiren MF, Liang Y-C. A discrete particle swarm optimization algorithm for the no-wait flowshop scheduling problem. *Computers & Operations Research*, 2007, in press, doi:10.1016/j.cor.2006.12.030.
- [12] Trelea IC. The particle swarm optimization algorithm: convergence analysis and parameter selection. *Information Processing Letters* 2003;85:317–25.
- [13] Clerc M. Discrete particle swarm optimization, illustrated by the traveling salesman problem. *New optimization techniques in engineering*. Berlin: Springer; 2004 p. 219–39.
- [14] Poli R, Kennedy J, Blackwell T. Particle swarm optimization. An overview. *Swarm intelligence*. Berlin: Springer; 2007 [Chapter 5].
- [15] Eberhart RC., Shi, Y. Comparing inertia weights and constriction factors in particle swarm optimization. In: IEEE international conference on evolutionary computation, 2001, p. 81–6.
- [16] He S, Wu QH, Wen JY, Saunders JR, Paton RC. A particle swarm optimizer with passive congregation. *Biosystems* 2004;78:135–47.
- [17] Shi Y, Eberhart RC. Parameter selection in particle swarm optimization. In: Proceedings 7th annual conference on evolutionary programming; 1998, p. 591–600.
- [18] Clerc M. Particle swarm optimization. *ISTE*; 2006.
- [19] Zhang W, Liu Y, Clerc M. An adaptive PSO algorithm for reactive power optimization. In: Sixth international conference on advances in power control, operation and management, Hong Kong; 2003.
- [20] Parrish JK, Hamner WM. Animal groups in three dimensions. Cambridge, UK: Cambridge University Press; 1997.
- [21] Xie XF, Zhang WJ, Yang ZL. A dissipative particle swarm optimization. In: IEEE congress on evolutionary computation (CEC'02), HI, USA; 2002.
- [22] Fieldsend JE., Singh S. A multi-objective algorithm based upon particle swarm optimisation, an efficient data structure and turbulence. In: Proceedings of U.K. workshop on computational intelligence (UKCI'02), United Kingdom; 2002. p. 37–44.
- [23] Hu X, Eberhart RC. Tracking dynamic systems with PSO: where's the cheese? In: Proceedings of the workshop on particle swarm optimization. Indianapolis, USA; 2001.
- [24] Parsopoulos KE, Vrahatis MN. Particle swarm optimizer in noisy and continuously changing environments. In: Hamza MH, editor, *Artificial intelligence and soft computing*; 2001. p. 289–94.
- [25] Parsopoulos KE, Vrahatis MN. On the computation of all global minimizers thorough particle swarm optimization. *IEEE Transactions on Evolutionary Computation* 2004;8:211–24.
- [26] Loøvbjerg M, Krink T. Extending particle swarms with self-organized criticality. In: Proceedings of the IEEE congress on evolutionary computation; 2002. p. 1588–93.
- [27] Blackwell T, Bentley PJ. Don't push me! Collision-avoiding swarms. In: Proceedings of the IEEE congress on evolutionary computation; 2002. p. 1691–96.
- [28] Robinson J, Sinton S, Rahmat-Samii Y. Particle swarm, genetic algorithm, and their hybrids: optimization of a profiled corrugated horn antenna. In: IEEE swarm intelligence symposium; 2002. p. 314–7.
- [29] Angeline PJ. Evolutionary optimization versus particle swarm optimization: philosophy and performance differences. In: Proceedings of evolutionary programming VII; 1998. p. 601–10.
- [30] Hendtlass T. A combined swarm differential evolution algorithm for optimization problems. In: *Lecture notes in computer science*, vol. 2070; 2001. p. 11–8.
- [31] Mendes R. Population topologies and their influence in particle swarm performance. PhD thesis, Departamento de Informatica, Escola de Engenharia, Universidade do Minho; 2004.
- [32] Liang JJ., Suganthan PN. Dynamic multiswarm particle swarm optimizer (DMS-PSO). In: Proceedings SIS 2005 IEEE swarm intelligence; 2005. p. 124–9.
- [33] Corominas A, Kubiak W, Moreno N. Response time variability. *Journal of Scheduling* 2007;10:97–110.
- [34] Monden Y. Toyota production systems. Norcross, GA: Industrial Engineering and Management Press; 1983.
- [35] Dong L, Melhem R, Mossel D. Time slot allocation for real-time messages with negotiable distance constraint requirements. In: *Real-time technology and application symposium, RTAS*, Denver; 1998.

- [36] Waldspurger CA, Weihl WE. Stride scheduling: deterministic proportional-share resource management. Technical Memorandum MIT/LCS/TM-528, MIT, Laboratory for Computer Science, Cambridge; 1995.
- [37] Anily S, Glass CA, Hassin R. The scheduling of maintenance service. *Discrete Applied Mathematics* 1998;82:27–42.
- [38] García A, Pastor R, Corominas A. Solving the response time variability problem by means of metaheuristics. *Artificial Intelligence Research and Development* 2006;146:187–94.
- [39] Clerc M, Kennedy J. The particle swarm: explosion, stability, and convergence in a multidimensional complex space. *IEEE Transactions on Evolutionary Computation* 2002;6:58–73.
- [40] Shi Y, Eberhart RC. A modified particle swarm optimizer. In: *IEEE international conference of evolutionary computation*; 1998b. p. 69–73.
- [41] Eiben AE, Hinterding R, Michalewicz Z. Parameter control in evolutionary algorithms. *IEEE Transactions on Evolutionary Computation* 1999;3:124–41.
- [42] Angeline PJ. Two self-adaptive crossover operators for genetic programming. *Advances in genetic programming*, vol. 2; 1996. p. 89–109.
- [43] Hinterding R, Michalewicz Z, Peachey TC. Self-adaptive genetic algorithm for numeric functions. *Lecture notes in computer science* 1996;1141:420–9.
- [44] Corominas A, Kubiak W, Pastor R. Solving the response time variability problem (RTVP) by means of mathematical programming. Working paper IOC-DT, Universitat Politècnica de Catalunya, Spain; 2006.
- [45] Adenso-Díaz B, Laguna M. Fine-tuning of algorithms using fractional experimental designs and local search. *Operations Research* 2006;54:99–114.